

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA0704

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

SOLUTIONS:

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

AIM:

To develop a Python program to simulate the CSMA/CD protocol in a star topology consisting of four hosts connected through a central switch. The program detects collisions, calculates random back-off intervals, retries transmission, records the events in `csma_log.txt`, and displays the star topology graphically.

PYTHON CODE:

```
import random
import matplotlib.pyplot as plt

random.seed(10)

hosts=["Host1","Host2","Host3","Host4"]
macs=["AA:BB:CC:01","AA:BB:CC:02","AA:BB:CC:03","AA:BB:CC:04"]

pending={h:True for h in hosts}
attempts={h:0 for h in hosts}
collisions={h:0 for h in hosts}
backoff={h:0 for h in hosts}

log=[]
round_no=0

while any(pending.values()):
    round_no+=1

    for h in hosts:
        if pending[h] and backoff[h]>0:
            backoff[h]-=1

    ready=[h for h in hosts if pending[h] and backoff[h]==0]

    if not ready:
        log.append(f"Round {round_no}: All active hosts are in back-off")
        continue

    for h in ready:
        attempts[h]+=1
```

```

log.append(f'\nRound {round_no}')
log.append("Transmission attempts: "+, ".join(ready))

if len(ready)>1:
    log.append("COLLISION DETECTED at CENTRAL SWITCH")

    for h in ready:
        collisions[h]+=1
        window=min(2**collisions[h]-1,15)
        wait=random.randint(0,window)
        backoff[h]=wait
        log.append(f'{h}: Random Back-off = {wait} slot(s)')
    else:
        h=ready[0]
        pending[h]=False
        log.append(f'{h}: Transmission successful')

with open("csma_log.txt","w") as f:
    f.write("\n".join(log))

print("STAR TOPOLOGY - CSMA/CD SIMULATION")
print("="*45)

print("\nNETWORK TOPOLOGY")
print("      HOST1")
print("      |")
print("      |")
print("HOST2 ----- CENTRAL SWITCH ----- HOST3")
print("      |")
print("      |")
print("      HOST4")

print("\nTRANSMISSION LOG")
print("="*45)

for line in log:
    print(line)

print("\nFINAL TRANSMISSION DETAILS")
print("="*45)

for h in hosts:
    print(f'{h} ( {macs[hosts.index(h)]} )')
    print(f'Attempts : {attempts[h]}')
    print(f'Collisions: {collisions[h]}')
    print(f'Status   : Transmission Completed')
    print()

print("Log file created: csma_log.txt")

plt.figure(figsize=(10,7))

positions={

```

```

"Host1":(0,2),
"Host2":(-2,0),
"Switch":(0,0),
"Host3":(2,0),
"Host4":(0,-2)
}

for host in hosts:
    x1,y1=positions[host]
    x2,y2=positions["Switch"]
    plt.plot([x1,x2],[y1,y2],linewidth=3)

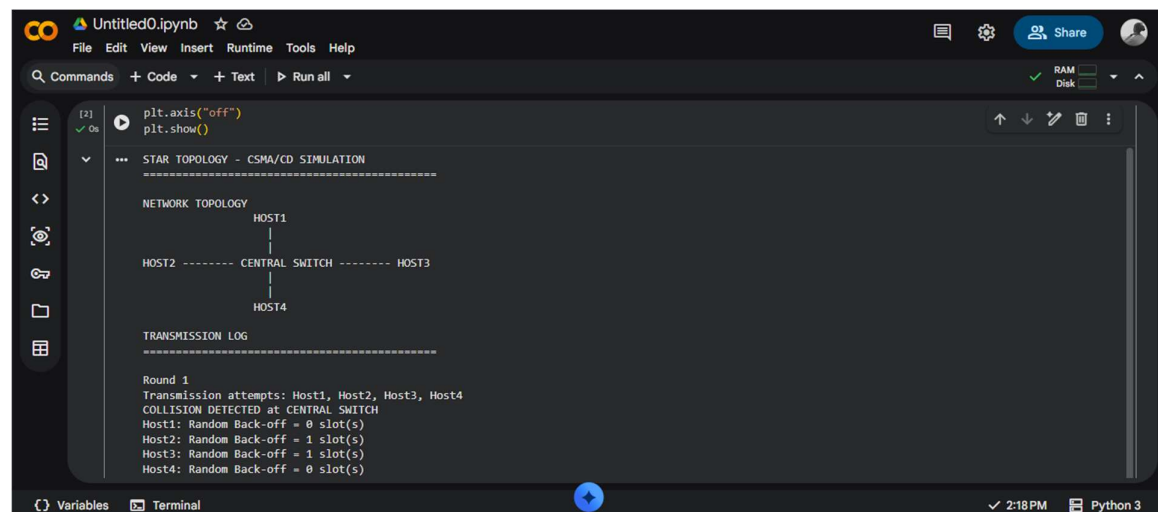
plt.scatter(0,0,s=2500)
plt.scatter([0,-2,2,0],[2,0,0,-2],s=1800)

plt.text(0,0,"CENTRAL\nSWITCH",ha="center",va="center",fontsize=11)
plt.text(0,2,"HOST 1",ha="center",va="center",fontsize=11)
plt.text(-2,0,"HOST 2",ha="center",va="center",fontsize=11)
plt.text(2,0,"HOST 3",ha="center",va="center",fontsize=11)
plt.text(0,-2,"HOST 4",ha="center",va="center",fontsize=11)

plt.title("Star Topology - CSMA/CD Simulation",fontsize=15)
plt.xlim(-3,3)
plt.ylim(-3,3)
plt.axis("off")
plt.show()

```

OUTPUT:



```
Untitled0.ipynb ☆
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
RAM Disk
[2] ✓ Os
plt.axis("off")
plt.show()
...
Round 2
Transmission attempts: Host1, Host2, Host3, Host4
COLLISION DETECTED at CENTRAL SWITCH
Host1: Random Back-off = 1 slot(s)
Host2: Random Back-off = 3 slot(s)
Host3: Random Back-off = 3 slot(s)
Host4: Random Back-off = 2 slot(s)

Round 3
Transmission attempts: Host1
Host1: Transmission successful

Round 4
Transmission attempts: Host4
Host4: Transmission successful

Round 5
Transmission attempts: Host2, Host3
COLLISION DETECTED at CENTRAL SWITCH
Host2: Random Back-off = 2 slot(s)
Host3: Random Back-off = 0 slot(s)

...

Round 6
Transmission attempts: Host3
Host3: Transmission successful

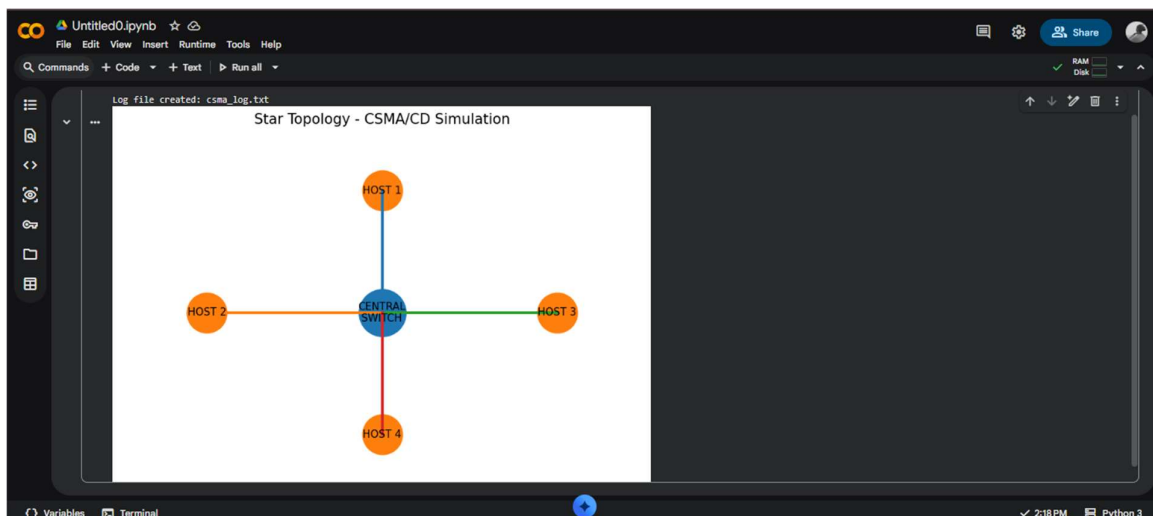
Round 7
Transmission attempts: Host2
Host2: Transmission successful

FINAL TRANSMISSION DETAILS
=====
Host1 (AA:BB:CC:01)
Attempts : 3
Collisions: 2
Status : Transmission Completed

Host2 (AA:BB:CC:02)
Attempts : 4
Collisions: 3
Status : Transmission Completed

Host3 (AA:BB:CC:03)
Attempts : 4
Collisions: 3
Status : Transmission Completed
```

GRAPHICAL REPRESENTATION:



TRANSMISSION / COLLISION / BACK-OFF DETAILS

Host MAC Address Function

Host 1 AA:BB:CC:01 Sends frames through switch

Host 2 AA:BB:CC:02 Sends frames through switch

Host 3 AA:BB:CC:03 Sends frames through switch

Host 4 AA:BB:CC:04 Sends frames through switch

The program records:

- Transmission attempts
- Collision events
- Random back-off intervals
- Successful transmissions
- Final transmission statistics

All events are also stored in **csma_log.txt**.

FINAL EXPLANATION:

The program simulates four hosts connected to a central switch in a **star topology**. When multiple hosts attempt transmission at the same time, a collision is detected. The affected hosts calculate random back-off intervals and wait before attempting transmission again. The process continues until all hosts successfully transmit. The transmission attempts, collisions and back-off intervals are recorded in **csma_log.txt**. This demonstrates how CSMA/CD helps reduce transmission conflicts in Ethernet communication.

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

AIM:

To develop a Python client-server application using the **socket library** to simulate communication between two hosts connected through a central switch in a star topology. The program maintains a simulated MAC Address Table, learns MAC addresses and forwards frames to the appropriate port.

PHYTON CODE:

```
import socket
import threading
import matplotlib.pyplot as plt

mac_table={}
connections={}

```



```

ports={}

def switch_server(server1,server2):

    clients=[server1,server2]

    for port,conn in enumerate(clients,1):

        data=conn.recv(1024).decode()
        mac=data.split("|")[1]

        mac_table[mac]=port
        connections[mac]=conn
        ports[mac]=port

        print(f'Host {mac} connected to Switch Port {port}')

    print("\nMAC ADDRESS TABLE")
    print("="*35)

    for mac,port in mac_table.items():
        print(f'{mac} -> Port {port}')

    print("="*35)

    server1.send(b"READY")
    server2.send(b"READY")

    data=server1.recv(1024).decode()

    parts=data.split("|",3)

    source=parts[1]
    destination=parts[2]
    message=parts[3]

    print("\nFRAME TRANSMISSION")
    print("="*35)
    print(f'Source MAC : {source}')
    print(f'Destination MAC : {destination}')
    print(f'Message : {message}')

    if destination in connections:

        destination_port=ports[destination]
        destination_socket=connections[destination]

        print("\nMAC ADDRESS LEARNED")
        print(f'{destination} -> Port {destination_port}')

        print("\nFRAME FORWARDING")
        print(f'Forwarding frame through Port {destination_port}')

        destination_socket.send(

```

```

        f"FORWARDED|{source}|{destination}|{message}".encode()
    )

    print("Frame forwarded successfully")

def host1(client):

    client.send(b"REGISTER|AA:BB:CC:01")
    client.recv(1024)

    client.send(
        b"FRAME|AA:BB:CC:01|AA:BB:CC:02|Hello from Host 1"
    )

def host2(client):

    client.send(b"REGISTER|AA:BB:CC:02")
    client.recv(1024)

client1,server1=socket.socketpair()
client2,server2=socket.socketpair()

server_thread=threading.Thread(
    target=switch_server,
    args=(server1,server2)
)

server_thread.start()

thread2=threading.Thread(
    target=host2,
    args=(client2,)
)

thread1=threading.Thread(
    target=host1,
    args=(client1,)
)

thread2.start()
thread1.start()

thread2.join()
thread1.join()
server_thread.join()

data=client2.recv(1024).decode()

parts=data.split("|",3)

print("\nFRAME DELIVERY")
print("="*35)
print(f'Source      : {parts[1]}')
print(f'Destination : {parts[2]}')

```

```

print(f'Message    : {parts[3]}')
print("="*35)

print("\nSTAR TOPOLOGY GRAPHICAL REPRESENTATION")
print("="*35)

plt.figure(figsize=(9,7))

plt.plot([0,1],[1,2.5],linewidth=3)
plt.plot([2,1],[1,2.5],linewidth=3)

plt.scatter([0,2],[1,1],s=1800)
plt.scatter([1],[2.5],s=2500)

plt.text(0,1,"HOST 1\nAA:BB:CC:01",
        ha="center",va="center",fontsize=10)

plt.text(2,1,"HOST 2\nAA:BB:CC:02",
        ha="center",va="center",fontsize=10)

plt.text(1,2.5,"CENTRAL\nSWITCH",
        ha="center",va="center",fontsize=11)

plt.text(0.3,1.65,"Port 1",fontsize=10)
plt.text(1.45,1.65,"Port 2",fontsize=10)

plt.annotate(
    "Frame",
    xy=(1.7,1.65),
    xytext=(1.25,1.2),
    arrowprops=dict(arrowstyle="->",linewidth=2)
)

plt.title(
    "Star Topology - MAC Learning and Frame Forwarding",
    fontsize=14
)

plt.xlim(-0.7,2.7)
plt.ylim(0.4,3)
plt.axis("off")
plt.show()

client1.close()
client2.close()
server1.close()
server2.close()

```

OUTPUT:

```
Host AA:BB:CC:01 connected to Switch Port 1
Host AA:BB:CC:02 connected to Switch Port 2

MAC ADDRESS TABLE
=====
AA:BB:CC:01 -> Port 1
AA:BB:CC:02 -> Port 2
=====

FRAME TRANSMISSION
=====
Source MAC : AA:BB:CC:01
Destination MAC : AA:BB:CC:02
Message : Hello from Host 1

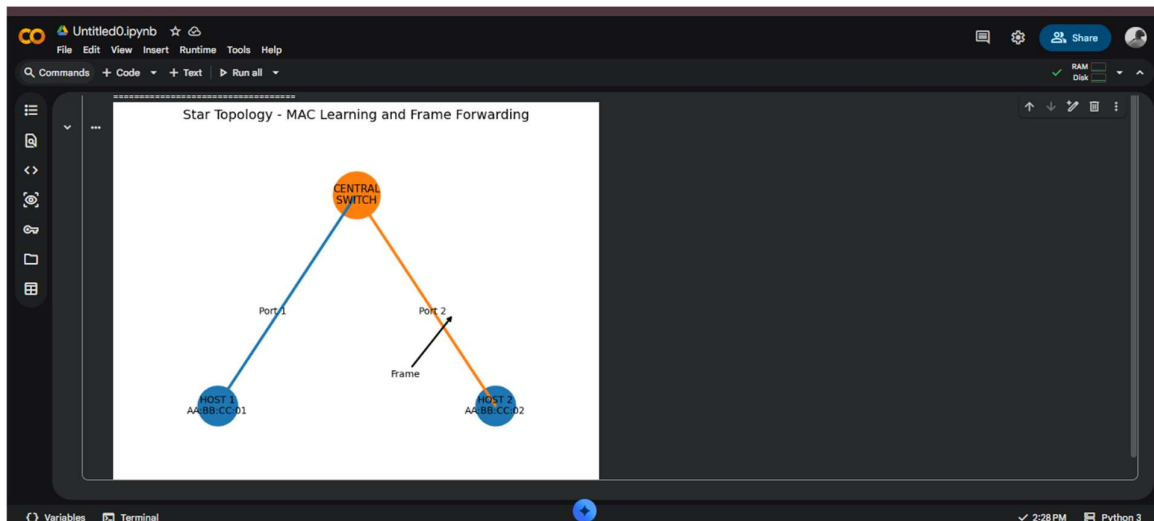
MAC ADDRESS LEARNED
AA:BB:CC:02 -> Port 2

FRAME FORWARDING
Forwarding frame through Port 2
Frame forwarded successfully

FRAME DELIVERY
=====
Source : AA:BB:CC:01
Destination : AA:BB:CC:02
Message : Hello from Host 1
=====

STAR TOPOLOGY GRAPHICAL REPRESENTATION
=====
Star Topology - MAC Learning and Frame Forwarding
```

GRAPHICAL REPRESENTATION:

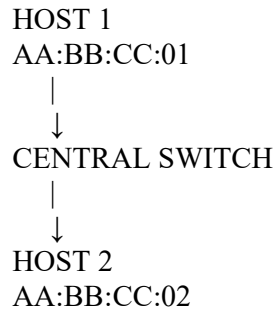


MAC ADDRESS TABLE / FRAME FORWARDING DETAILS:

MAC Address Switch Port

AA:BB:CC:02 Port 1

AA:BB:CC:01 Port 2

Frame:

The switch learns the MAC address of each connected host and stores the MAC-to-port association in its simulated MAC Address Table.

FINAL EXPLANATION:

The program simulates a client-server communication system using Python's socket library. Two hosts are connected through a central switch in a star topology. When the hosts register with the switch, their MAC addresses are learned and stored with their respective ports. When Host 1 sends a frame to Host 2, the switch searches the MAC Address Table, identifies the destination port and forwards the frame through that port. This demonstrates the basic operation of MAC learning and frame forwarding in an Ethernet switch.